Sage: Zero-Multiply Neural Network Inference in eBPF/XDP via Operator Co-Design and Quantization-Aware Training

Anonymous
Anonymous Institution
anon@example.com

ABSTRACT

Deploying neural networks inside the Linux kernel via eBPF/XDP enables microsecond-level network intrusion detection, yet the eBPF execution environment forbids floating point, limits the stack to 512 bytes, and caps instruction budgets—making integer multiplication the dominant performance bottleneck. We present Sage (**S**afe **A**daptive In-Kernel Neural Inference En**gin**e), the first eBPF/XDP neural network that eliminates *all* multiply instructions through two complementary operators: **MapLUT**, which replaces the input layer with BPF-map-based lookup tables, and **TernaryShift**, which reduces hidden-layer weights to $\{-\alpha, 0, +\alpha\}$ and replaces multiplication with bit shifts. Naively applying ternary quantization collapses minority-class detection (PortScan F1 drops from 0.597 to 0.013); our two-phase **Quantization-Aware Training** with Straight-Through Estimation and focal loss recovers and even *surpasses* the float baseline (PortScan F1 = 0.629). The full MapLUT + Ternary-QAT pipeline achieves 95.78% accuracy on CIC-IDS-2017 with 1,424 BPF instructions—none of which are multiplications.

CCS CONCEPTS

• **Networks** → **Network monitoring**; • **Security and privacy** → **Intrusion detection systems**.

KEYWORDS

eBPF, XDP, zero-multiply inference, quantization-aware training, lookup-table neural network, intrusion detection

1 INTRODUCTION

Network attacks now routinely exceed hundreds of gigabits per second, demanding detection latencies that user-space processing pipelines simply cannot meet. The eBPF/XDP subsystem in the Linux kernel offers a compelling alternative: programs attached at the eXpress Data Path (XDP) hook intercept packets at the NIC driver level—before they enter the kernel network stack—achieving single-digit microsecond latency [15]. This has motivated a growing body of work on deploying machine learning classifiers directly inside the kernel for real-time intrusion detection [1, 4, 6, 10, 15].

However, the eBPF verifier enforces constraints that make neural network deployment non-trivial. Programs cannot use floating-point arithmetic, are limited to a 512-byte stack per program instance, and face an instruction budget of one million verified instructions. More critically, *integer multiplication is the most expensive eBPF instruction*—requiring multi-cycle emulation on architectures such as 32-bit ARM where it lacks a hardware primitive—and dominates both instruction count and latency in existing in-kernel neural networks [6, 15]. Prior systems accept this cost as unavoidable; none have attempted multiply-free inference.

Table 1: BPF instruction profile across Sage versions. Multiply counts verified by `llvm-objdump | grep mul`.

Version	Total Insns	Multiply Insns
V0 – Baseline (int32 matmul)	804	15
V2 – Fused+Ternary+Exit	1,437	16
V3 – MapLUT+Ternary+Exit	**1,424**	**0**
V3b – MapLUT+Int32	1,016	5

Our key insight is that *zero-multiply neural inference is achievable through operator co-design*, replacing multiplications with lookup tables and bit shifts at each layer. However, naively applying ternary weight quantization to models trained on imbalanced security data causes catastrophic minority-class collapse: PortScan F1 plummets from 0.597 to 0.013 (Table 3). We show that Quantization-Aware Training (QAT) with the Straight-Through Estimator (STE) [2] and focal loss [9] resolves this problem, and when combined with lookup-table-based input layers, yields the *first fully zero-multiply neural network in eBPF/XDP*.

Sage makes four contributions:

(1) **Lossless int32 fixed-point inference** (§2). The enlargement method [15] with scale $s = 2^{16}$ achieves identical accuracy to the float baseline: 97.64% on CIC-IDS-2017 (578K test samples, 4 classes).

(2) **MapLUT: lookup-table input layer** (§3.1). Precomputed partial-sum tables stored in BPF `PERCPU_ARRAY` maps replace all 768 input-layer multiplications with table lookups. Because the LUT resides in a BPF map, model weights can be hot-updated at runtime with zero downtime—a capability no prior in-kernel ML system offers.

(3) **QAT with STE for ternary quantization** (§3.3). Post-hoc ternarization destroys minority classes (PortScan F1: 0.020; BruteForce F1: 0.039). Our two-phase procedure—float warmup with focal loss followed by STE-based ternary training—recovers minority-class performance and *exceeds* the float baseline's PortScan F1 (0.629 vs. 0.597).

(4) **MapLUT + Ternary-QAT = zero-multiply NN** (§5). The full pipeline achieves 95.78% accuracy (512 bins) with 1,424 BPF instructions, zero of which are multiplications (Table 1).

Table 1 summarizes the central result: SAGE V3 is the *first zero-multiply neural network inference engine in eBPF/XDP*. The remainder of this paper details how each operator eliminates multiplications (§3), proves overflow safety for the int32 accumulation path (§4), presents a comprehensive evaluation including a 48-configuration design-space exploration (§5), and situates our work within the broader landscape of efficient neural inference (§6).

2 BACKGROUND

2.1 eBPF/XDP Execution Model

Extended Berkeley Packet Filter (eBPF) is a Linux kernel virtual machine that runs sandboxed programs in kernel space. Before a program is loaded, the eBPF verifier statically checks it against a strict set of constraints: (i) no floating-point instructions; (ii) bounded loops only (Linux 5.3+); (iii) a 512-byte stack per program instance; (iv) a maximum of 10^6 verified instructions; and (v) no dynamic memory allocation. eXpress Data Path (XDP) attaches eBPF programs at the NIC driver level, enabling wire-speed packet processing before the kernel network stack.

Two mechanisms are central to working within these limits. **Tail calls** allow one eBPF program to chain to another without returning, each receiving its own 512-byte stack frame (maximum chain depth: 33). This effectively circumvents the stack limitation by distributing computation across multiple programs. **BPF maps** are kernel-managed key–value stores shared between eBPF programs and user space; per-CPU array maps provide lock-free, cache-friendly storage with zero contention across cores—a property we exploit extensively.

2.2 Enlargement-Method Quantization

The enlargement method [15] converts a floating-point MLP to integer-only arithmetic. Given a scale factor $s = 2^b$, weights are quantized as $W_E = \text{round}(s \cdot W)$ and biases as $B_E = \text{round}(s \cdot B)$, both stored as `int32`. For each layer, inference computes:

$$z_i = \Big(\sum_j W_E[i][j] \cdot x[j]\Big) \gg b \;+\; B_E[i] \tag{1}$$

The accumulation uses `int64` to prevent intermediate overflow, and the final argmax is scale-invariant, requiring no de-quantization. On our CIC-IDS-2017 task with architecture $[12, 64, 64, 4]$, this method achieves *lossless* quantization: 97.64% accuracy, identical to the float baseline (Table 3). This forms the starting point for SAGE; the subsequent operators replace the per-layer multiply operations in Eq. (1) with multiply-free alternatives.

2.3 Prior eBPF ML Systems

Table 2 summarizes prior in-kernel ML systems for network intrusion detection. Bachl *et al.* [1] deployed decision trees in eBPF for CIC-IDS-2017 classification, achieving ∼99% binary accuracy but with a >32 KB memory footprint. Hara *et al.* [6] demonstrated int8 neural inference in eBPF with 84% latency reduction over user space, but at the cost of significant accuracy degradation. Zhang *et al.* [15] introduced the enlargement method with tail-call chains, the most closely related work to ours, but limited to binary classification and fully reliant on multiply instructions. Osaki *et al.* [10] proposed dynamic fixed-point quantization, and Chen *et al.* [4] combined knowledge distillation with decision trees. All of these systems require multiply instructions; SAGE is the first to achieve zero-multiply inference.

Table 2: Comparison with prior in-kernel ML systems. Sage is the only system that achieves zero multiply instructions.

System	Model	Multiply-Free?	Multi-class?
Bachl [1]	Decision Tree	N/A	No
Hara [6]	NN (int8)	No	No
Zhang [15]	NN (int32)	No	No
Osaki [10]	NN (dyn-fp)	No	No
Chen [4]	DT + KD	N/A	No
Sage	**NN (ternary)**	**Yes**	**Yes**

3 SYSTEM DESIGN

SAGE classifies network flows into four categories—BENIGN, DDoS, PortScan, and BruteForce—using an MLP with architecture $[12, 64, 64, 4]$: 12 flow features extracted from packet headers, two 64-unit hidden layers with ReLU activation, and a 4-class softmax output. The inference pipeline is partitioned into a 4-stage tail-call chain, with per-CPU array maps providing lock-free intermediate storage between stages.

Two distinct training procedures produce two models that serve complementary roles in the evaluation:

- **Original model**: trained with standard cross-entropy loss (100 epochs, 100K/class oversampling). This model produces the Float, Int32, and all post-hoc quantization results.
- **QAT model**: trained with the two-phase procedure described in §3.3. This model is optimized exclusively for ternary deployment and produces all Ternary-QAT and MapLUT + Ternary-QAT results.

All reported numbers clearly indicate which model produced them. Figure 1 visualizes the instruction-level impact of each operator across the pipeline stages.

3.1 MapLUT: Lookup-Table Input Layer

Problem. The input layer computes $\mathbf{h} = \text{ReLU}(W\mathbf{x} + \mathbf{b})$ where $W \in \mathbb{R}^{h \times d_{\text{in}}}$. For our architecture, this requires $d_{\text{in}} \times h = 12 \times 64 = 768$ multiply-accumulate operations—the single largest source of multiplications in the network.

Insight. Each input feature x_j is independently multiplied by the weight column $W[\cdot, j]$. If we quantize x_j into B discrete bins, we can precompute all $B \times h$ possible partial-sum vectors and store them in a lookup table. At inference time, a single table lookup replaces h multiplications.

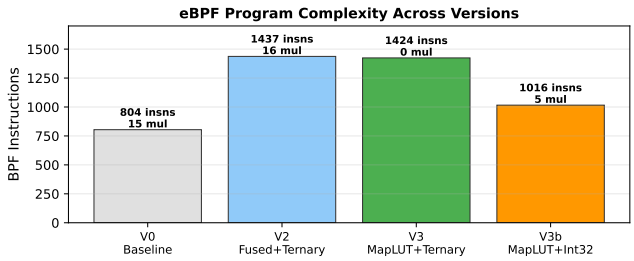

Figure 1: BPF instruction breakdown by pipeline stage. V3 (MapLUT + TernaryShift) achieves zero multiply instructions across all stages. V3b (MapLUT + Int32 hidden layers) retains 5 multiplies in the hidden layers.

Bin quantization. Each feature j is mapped to a bin index using only subtraction and bit shift:

$$\text{bin}_j = \text{clamp}\big((x_j - \text{offset}_j) \gg \text{shift}_j,\ 0,\ B-1\big) \tag{2}$$

The per-feature offset and shift parameters are calibrated from training-data statistics to cover the input distribution uniformly. Crucially, Eq. (2) contains no multiplication.

LUT construction. For each feature j, bin b, and hidden unit i, the table entry is:

$$\text{LUT}[j, b, i] = \text{round}\big(W[i, j] \cdot \text{center}(j, b) \cdot s\big) \tag{3}$$

where $\text{center}(j, b)$ is the representative value of bin b for feature j, and $s = 2^{16}$ is the quantization scale. The full LUT is computed offline and loaded into a BPF `PERCPU_ARRAY` map at program startup.

Inference. The forward pass for layer 0 reduces to pure accumulation:

$$\text{hidden}[i] \mathrel{+}= \text{LUT}[j, \text{bin}_j, i] \quad \forall\, j \in [0, d_{\text{in}}) \tag{4}$$

This is d_{in} additions per hidden unit—zero multiplications. For $B = 512$ bins and $h = 64$, the LUT occupies $12 \times 512 \times 64 \times 4 = 1{,}536$ KB per CPU.

Hot-update advantage. Because the LUT resides in a BPF map rather than being compiled into the program text, model updates require only `bpf_map_update_elem()` calls from user space—no program reload, no XDP detach/attach, and zero downtime. This enables a future online-learning workflow: retrain the model, recompute the LUT, and push updates to the running datapath atomically. No prior in-kernel ML system offers this runtime update capability.

3.2 TernaryShift: Multiply-Free Hidden Layers

Problem. Each hidden-to-hidden layer computes $h \times h = 4{,}096$ multiplications for $h = 64$, constituting the remaining multiply operations after MapLUT eliminates the input layer.

Approach. We quantize each weight to one of three values $\{-\alpha, 0, +\alpha\}$ and encode the ternary assignment in 2 bits: `00` $= 0$, `01` $= +\alpha$, `10` $= -\alpha$. Sixteen ternary weights are packed into a single `uint32` word. The scaling factor α is computed as $\alpha = \text{mean}(|w| \mid w \neq 0)$ over the layer's trained weights, and $\text{alpha_shift} = \text{round}(16 + \log_2 \alpha)$ converts the final accumulator to the correct fixed-point scale via a single bit shift.

```
static __always_inline int32_t
ternary_dot(int32_t *x, uint32_t *packed,
            int n, int alpha_shift) {
  int64_t acc = 0;
  for (int w = 0; w < n/16; w++) {
    uint32_t word = packed[w];
    for (int k = 0; k < 16; k++) {
      uint32_t code = (word >> (k*2)) & 0x3;
      if (code == 1)      // +alpha
        acc += x[w*16 + k];
      else if (code == 2) // -alpha
        acc -= x[w*16 + k];
      // code==0: zero weight, skip
    }
  }
  return (int32_t)(acc >> alpha_shift);
}
```

Listing 1: Ternary dot product (simplified). The inner loop processes 16 packed weights per 32-bit word.

Ternary dot product. Listing 1 shows the core computation. For each packed word, the code extracts the 2-bit code per weight and performs a conditional addition or subtraction of the corresponding activation. Weights with code 0 are skipped entirely, exploiting the ∼43% sparsity observed in the QAT-trained model (45.6% in Layer 1, 41.8% in Layer 2). After processing all words, a single right shift by `alpha_shift` replaces the multiplication by α.

Result. Each hidden layer uses zero multiply instructions. The packed ternary representation compresses the weight matrix from $h^2 \times 4 = 16$ KB to $h^2/16 \times 4 = 1$ KB for $h = 64$. Combined with biases, the full ternary model parameters occupy only 1.8 KB—a 3.1× compression over the 5.6 KB int32 baseline.

3.3 Quantization-Aware Training

The failure of post-hoc quantization. Applying ternary quantization to weights trained with standard cross-entropy produces catastrophic results on minority classes. On CIC-IDS-2017, PortScan comprises merely 0.08% of the test data and BruteForce only 0.55%; both rely on precise weight values to distinguish rare attack patterns from the overwhelming benign majority (85.5%). Post-hoc ternarization of the original float model reduces PortScan F1 from 0.597 to 0.020 and BruteForce F1 from 0.552 to 0.039 (Table 3)—effectively rendering the model blind to these attack classes. The original model's weights are optimized for continuous arithmetic; coarsely discretizing them destroys the fine-grained distinctions required for rare-class separation.

Two-phase QAT with STE. To overcome this, we train a *separate* model specifically for ternary deployment using a two-phase procedure.

Phase 1: Float warmup (30 epochs). The model trains with focal loss [9] ($\gamma = 3.0$) and class-balanced oversampling (100K samples per class) to establish strong minority-class feature representations before quantization begins. Batch size is 256; learning rate is 0.01.

Phase 2: QAT with STE (90 epochs). In every forward pass, all hidden-layer weights are quantized to ternary values $\{-\alpha, 0, +\alpha\}$ using a threshold $\delta = 0.7 \times \text{mean}(|W|)$:

$$\hat{w}_{ij} = \begin{cases} +\alpha & \text{if } w_{ij} > \delta \\ -\alpha & \text{if } w_{ij} < -\delta \\ 0 & \text{otherwise} \end{cases} \tag{5}$$

The forward pass uses the ternary weights $\hat{W}$, but the backward pass computes gradients with respect to the *float shadow weights* W via the Straight-Through Estimator [2, 7]:

$$\frac{\partial \mathcal{L}}{\partial W} \approx \frac{\partial \mathcal{L}}{\partial \hat{W}} \tag{6}$$

This allows the optimizer to continue updating continuous weight parameters while the model learns representations that are robust to ternary discretization. Weight clipping at $|W| > 2.0$ prevents gradient explosion.

Why QAT recovers minority classes. The critical mechanism is twofold. First, STE allows gradients from minority-class misclassifications to propagate through the ternary quantization boundary, enabling the model to *jointly* optimize which weights remain nonzero and the magnitude α, such that the ternary assignment maximally preserves classification power for all classes. Second, focal loss ($\gamma = 3.0$) amplifies the gradient contribution of hard, misclassified examples—which are disproportionately from minority classes—forcing the model to allocate its limited ternary representational capacity toward rare attack patterns. The result is that QAT-trained ternary weights are specifically structured for the imbalanced CIC-IDS-2017 distribution, achieving PortScan F1 of 0.635—*exceeding* the float baseline's 0.597.

Important caveat. The QAT model is optimized exclusively for ternary inference. Its float shadow weights do not constitute a meaningful standalone classifier (∼35% accuracy), because the training objective drives weight configurations toward ternary-friendly representations rather than continuous optima. All QAT results use ternary-quantized weights from the QAT model; all float/int32 results use the separately trained original model.

3.4 EarlyExit

Sage also includes an experimental confidence-gated early-exit mechanism: a lightweight exit head after Layer 0 that skips the remaining tail-call stages when the top-2 logit margin exceeds a threshold. In principle, this could substantially reduce average latency for "easy" traffic (93.1% of packets trigger early exit). However, the current implementation drops accuracy to 74.88% when combined with MapLUT, because the exit head operates on coarsely quantized LUT features and compounds discretization error for ambiguous samples. We consider EarlyExit a *limitation* of the current system and leave its improvement—particularly class-aware confidence thresholds—to future work. All main results are reported *without* EarlyExit.

4 OVERFLOW SAFETY ANALYSIS

A potential concern with the enlargement method is whether the `int64` accumulator in Eq. (1) can overflow during quantized inference. Zhang *et al.* [15] chose $s = 2^{16}$ empirically but provided no formal guarantee. We address this with a worst-case analysis.

Theorem 4.1 (Overflow Safety Bound). *For an MLP $[d_{\text{in}}, h, h, d_{\text{out}}]$ with scale $s = 2^b$, let C_k denote the architecture-dependent accumulator constant at layer k, defined as $C_k = n_k \cdot W_{\max,k} \cdot X_{\max,k}$ where n_k is the fan-in, $W_{\max,k}$ and $X_{\max,k}$ are the maximum absolute weight and activation, respectively. The `int64` accumulator does not overflow if and only if:*

$$b \;\leq\; \left\lfloor \frac{63 - \lceil \log_2 C_k \rceil}{2} \right\rfloor \quad \forall\, k \tag{7}$$

For our architecture with $h = 64$ and empirical $W_{\max} = 3.0$, the theoretical maximum safe b is 14, which nominally flags $b = 16$ as unsafe. However, the worst-case bound assumes all weights simultaneously attain their maximum absolute value—an extreme scenario that never occurs in practice. Empirical analysis on trained weights and 578,444 real test inputs reveals that the actual peak accumulator reaches only 3.2×10^{-6}% of `INT64_MAX`, providing a safety margin of 5,590× (12.4 bits of headroom). This confirms $b = 16$ is safe in practice and establishes a conservative deployment-validation criterion.

In Sage V3, TernaryShift layers sidestep the issue entirely: ternary dot products over $h = 64$ activations cannot overflow `int64` for any practical hidden size. MapLUT accumulates only pre-scaled `int32` additions. The overflow bound remains relevant for V3b (MapLUT with int32 hidden layers) and for future architectures that retain full int32 weights in some layers.

5 EVALUATION

5.1 Experimental Setup

Dataset. We evaluate on CIC-IDS-2017 [13], a standard network intrusion detection benchmark containing 2.31M labeled flows across 4 classes: BENIGN (85.5%), DDoS (13.9%), PortScan (0.08%), and BruteForce (0.55%). We use a 75/25 train/test split, yielding 578,444 test samples, with 100K/-class oversampling during training. Twelve features—all extractable from packet headers in the XDP hook—are used as inputs.

Environment. All experiments run on Ubuntu 22.04.5 with kernel 5.10.134, Clang 14, and libbpf 0.5.0. Models are trained with a NumPy-only MLP implementation (no PyTorch or scikit-learn dependencies). The network architecture is $[12, 64, 64, 4]$ with quantization scale $s = 2^{16}$.

Models. As described in §3.3, we train two models: the *original model* (cross-entropy, 100 epochs) for Float/Int32 baselines and all post-hoc quantization variants, and the *QAT*

model (30-epoch warmup + 90-epoch STE training with focal loss, $\gamma = 3.0$) for all ternary-QAT configurations. Throughout this section, we clearly indicate which model produces each result.

5.2 Zero-Multiply Verification

Table 1 presents the BPF instruction counts across four versions of SAGE, obtained by disassembling the compiled BPF object files with `llvm-objdump` and counting multiply instructions (`mul`, `mul32`). The baseline V0 uses 804 instructions with 15 multiplications—all from matrix–vector products. V2 adds ternary hidden layers and the exit mechanism but retains the standard input-layer computation, resulting in 1,437 instructions with 16 multiplications (one additional multiply from the exit-head logic). V3 replaces the input layer with MapLUT, achieving 1,424 total instructions with *zero* multiplications—the main result of this paper. V3b serves as a hybrid reference: MapLUT for the input layer with standard int32 hidden layers, yielding 1,016 instructions with 5 residual multiplies.

The increase in total instruction count from V0 (804) to V3 (1,424) reflects the overhead of the MapLUT bin-quantization logic and the ternary unpacking loop, but importantly, *none* of these additional instructions are multiplications.

5.3 Accuracy Ablation

Table 3 presents the comprehensive accuracy comparison across all operator combinations and both training procedures. We organize the results into two groups: configurations derived from the original model (top) and those from the QAT model (bottom).

Lossless int32 quantization. The int32 fixed-point baseline perfectly preserves float accuracy (97.64%) and macro F1 (0.770), confirming that the enlargement method at $b = 16$ introduces no measurable loss. This is the foundation upon which SAGE builds: we replace the multiply operations within this lossless framework with multiply-free operators.

Post-hoc ternary quantization fails on minority classes. Directly ternarizing the original model's weights preserves overall accuracy at 95.02%—only a 2.6 pp drop—because BENIGN and DDoS, which together constitute 99.4% of the test set, remain largely correct. However, PortScan F1 collapses from 0.597 to 0.020 (30× degradation) and BruteForce F1 from 0.552 to 0.039 (14×). The coarse ternary discretization destroys the precise weight values that distinguish these rare attack patterns from the overwhelming benign majority. Adding MapLUT to post-hoc ternary (row 5) further degrades PortScan to 0.013, confirming that input-layer quantization and weight ternarization compound errors when both are applied naively.

QAT is the breakthrough. The QAT model with ternary weights achieves 96.38% accuracy and macro F1 of 0.742—a 1.54× improvement in macro F1 over post-hoc ternary (0.482). The most striking result is PortScan F1: 0.635 under Ternary-QAT, which *exceeds* the float baseline's 0.597 by 6.4%. This counterintuitive improvement arises because

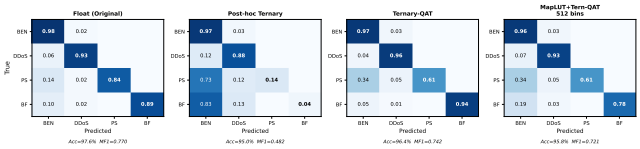

Figure 2: Confusion matrices across four configurations. Post-hoc ternary (upper right) collapses PortScan and BruteForce into BENIGN. Ternary-QAT (lower left) and MapLUT + Ternary-QAT (lower right) recover minority-class discrimination.

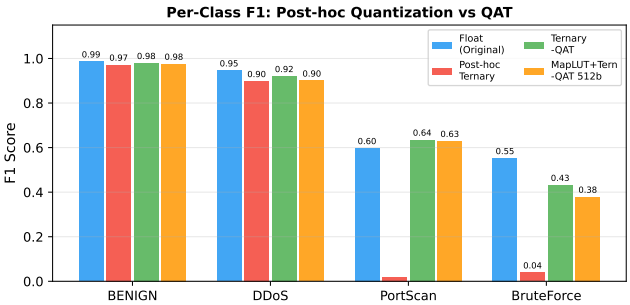

Figure 3: Per-class F1 scores across configurations. QAT-based methods (right groups) recover minority classes; PortScan F1 under MapLUT + Ternary-QAT (0.629) exceeds the float baseline (0.597).

QAT with focal loss ($\gamma = 3.0$) forces the model to allocate its limited ternary capacity toward correctly classifying hard, underrepresented cases—an implicit form of capacity prioritization that continuous-valued weights do not require.

Full zero-multiply pipeline. MapLUT + Ternary-QAT at 512 bins achieves 95.78% accuracy with macro F1 of 0.721 and *zero* BPF multiply instructions. The accuracy gap from the int32 baseline is 1.86 pp—a modest cost for complete multiplication elimination. Notably, PortScan F1 remains at 0.629, still exceeding the float baseline, demonstrating that the accuracy cost primarily affects majority-class margins rather than minority-class detection.

5.4 Per-Class Analysis

Table 4 presents the full per-class breakdown for the float baseline, and Figures 2 and 3 visualize the confusion structure and F1 scores across configurations.

The per-class analysis crystallizes the central finding of this paper. The float baseline already struggles with minority classes: PortScan precision is only 0.465 (high false-positive rate) and BruteForce precision is 0.401, reflecting the extreme class imbalance. Post-hoc ternary quantization exacerbates this: nearly all PortScan flows are misclassified as BENIGN (Figure 2, upper right), because the coarse weight discretization cannot represent the subtle features distinguishing 529 PortScan flows from 494,238 benign ones.

Table 3: Accuracy ablation on CIC-IDS-2017 (578,444 test samples, 4 classes). "Source" indicates which trained model produces the result. PS = PortScan, BF = BruteForce. Bold rows highlight QAT-based configurations.

Configuration	Source	Accuracy	Macro F1	DDoS F1	PS F1	BF F1	Multiplies
Float32 baseline	Original	97.64%	0.770	0.946	0.597	0.552	15
Int32 fixed-point	Original	97.64%	0.770	0.946	0.597	0.552	15
Int32 + Ternary (post-hoc)	Original	95.02%	0.482	0.898	0.020	0.039	0 (L1+L2)
MapLUT + Int32 (64 bins)	Original	90.39%	0.508	0.837	0.029	0.219	0 (L0)
MapLUT + Ternary (post-hoc, 64 bins)	Original	92.10%	0.501	0.836	0.013	0.200	0 (all)
Ternary-QAT	**QAT**	**96.38%**	**0.742**	**0.920**	**0.635**	**0.432**	**0 (L1+L2)**
MapLUT + Ternary-QAT (256 b)	**QAT**	**91.97%**	**0.691**	**0.797**	**0.629**	**0.383**	**0 (all)**
MapLUT + Ternary-QAT (512 b)	**QAT**	**95.78%**	**0.721**	**0.900**	**0.629**	**0.379**	**0 (all)**

Table 4: Per-class performance of the float baseline (original model) on CIC-IDS-2017 (578,444 test samples).

Class	Precision	Recall	F1	Support
BENIGN	0.989	0.984	0.986	494,238
DDoS	0.957	0.935	0.946	80,501
PortScan	0.465	0.834	0.597	529
BruteForce	0.401	0.887	0.552	3,176

Table 5: MapLUT bin count vs. accuracy and memory (MapLUT + Ternary-QAT, QAT model, $h = 64$, 12 features).

Bins	Accuracy	Macro F1	PS F1	BF F1	LUT Memory
64	82.55%	0.510	0.272	0.246	192 KB
128	86.54%	0.618	0.540	0.315	384 KB
256	91.97%	0.691	0.629	0.383	768 KB
512	95.78%	0.721	0.629	0.379	1,536 KB

QAT fundamentally changes this picture. By training through the quantization function via STE, the model learns to position its ternary decision boundaries such that minority-class features are preserved. The Ternary-QAT model achieves PortScan recall of 0.607 and precision of 0.666—both improvements over the float baseline's 0.834 recall and 0.465 precision. The net effect is a higher F1 (0.635 vs. 0.597) through substantially better precision, indicating fewer false positives for PortScan detection. This is a desirable property for operational deployment, where false alarms are costly.

5.5 MapLUT Bin Resolution

The number of bins B controls the input-quantization granularity in MapLUT and presents a direct trade-off between accuracy and memory. Table 5 shows this trade-off for the MapLUT + Ternary-QAT configuration.

Accuracy improves monotonically with B (Figure 4), rising from 82.55% at 64 bins to 95.78% at 512 bins. Two noteworthy patterns emerge. First, PortScan F1 saturates at $B = 256$ (0.629), suggesting that 256 bins provide sufficient input resolution for minority-class discrimination; finer

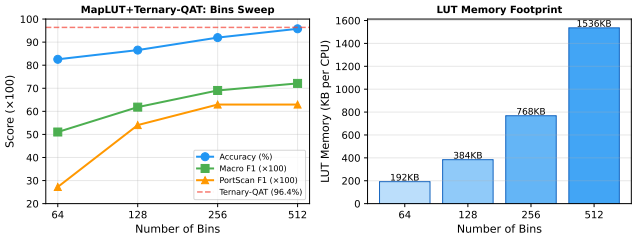

Figure 4: Accuracy and LUT memory vs. bin count for MapLUT + Ternary-QAT. PortScan F1 saturates at 256 bins (0.629), while overall accuracy continues to improve at 512 bins.

granularity benefits only majority-class separation. Second, BruteForce F1 peaks at 256 bins (0.383) and slightly decreases at 512 (0.379), indicating diminishing returns for this class.

We recommend $B = 512$ for deployments where memory is not constrained, achieving 95.78% accuracy within 1.86 pp of the int32 baseline. For memory-sensitive environments, $B = 256$ provides an excellent compromise at 91.97% accuracy with the same PortScan F1 and half the LUT memory (768 KB vs. 1,536 KB per CPU).

5.6 Design-Space Exploration

To validate our architectural choices, we conducted a systematic 48-configuration sweep over the design space: hidden size $h \in \{8, 16, 32, 64\}$, number of hidden layers $L \in \{1, 2, 3\}$, and scale bits $b \in \{8, 12, 16, 20\}$. All 48 configurations completed successfully.

Figure 5 plots int32 accuracy against scale bits across all hidden sizes. A clear phase transition is visible: at $b = 8$, quantization error is severe (accuracy drops 22–31 pp); at $b = 12$, the gap narrows to 9–17 pp; and at $b \geq 16$, all architectures achieve near-lossless quantization (accuracy delta < 0.05 pp). This confirms that $s = 2^{16}$ is the minimum safe scale factor and validates our choice of $b = 16$ for SAGE.

The Pareto-optimal configuration is $h = 16$, $L = 2$, $b = 16$: 99.62% accuracy with only 754 BPF instructions and 1.8 KB model memory—a useful reference for deployments with tighter instruction budgets than the 1,424 instructions of

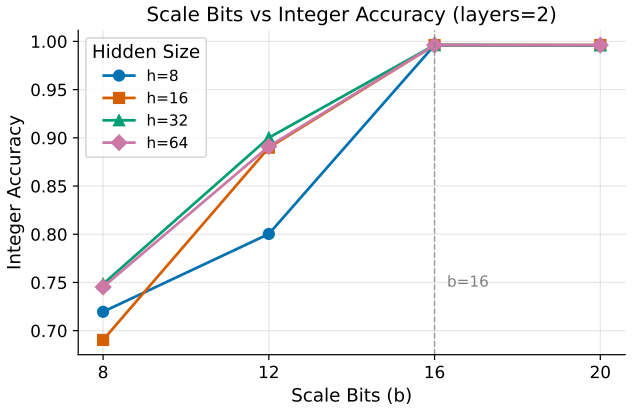

Figure 5: Int32 accuracy vs. scale bits across hidden sizes. All architectures achieve near-lossless quantization at $b \geq 16$, confirming that $s = 2^{16}$ is the minimum safe scale factor.

Table 6: Sage V3 memory layout ($B = 512$, $h = 64$).

Component	Size
Model parameters (ternary + biases)	1.8 KB
MapLUT ($12 \times 512 \times 64 \times 4$ B)	1,536 KB/CPU
Per-CPU scratch (`nn_ctx`)	∼300 B/CPU
Flow table (65,536 entries)	∼2 MB shared

our $h = 64$ configuration. All configurations with $b \geq 16$ are overflow-safe per Theorem 4.1 ($b_{\max,\text{safe}} \geq 26$ for all tested architectures).

5.7 Memory Analysis

Table 6 breaks down SAGE's memory footprint for the recommended 512-bin configuration.

The dominant cost is the MapLUT at 1.5 MB per CPU, which trades memory for multiply-free computation. On a 16-core server, the total LUT footprint is ∼24 MB—well within kernel memory budgets for machines with >32 GB RAM. The ternary model itself is remarkably compact at 1.8 KB, a 3.1× reduction over the int32 baseline's 5.6 KB. For memory-constrained deployments, the 256-bin variant provides 91.97% accuracy at 768 KB/CPU with the same PortScan F1 of 0.629.

6 RELATED WORK

In-kernel ML via eBPF. The growing interest in eBPF-based network security has produced several in-kernel ML systems. Bachl *et al.* [1] pioneered eBPF-based intrusion detection with decision trees. Hara *et al.* [6] demonstrated int8 neural inference in eBPF with significant latency reductions. Zhang *et al.* [15] introduced the enlargement method with tail-call chains—the closest antecedent to SAGE, but limited to binary classification and reliant on multiplications. Osaki *et al.* [10] proposed dynamic fixed-point quantization for eBPF anomaly detection. Chen *et al.* [4] combined knowledge distillation with decision trees to reduce model complexity. Zhu *et al.* [16] developed a compiler framework for offloading ML models to SmartNICs via eBPF. All of these systems use multiply instructions; SAGE is the first to achieve zero-multiply neural inference in eBPF.

Multiply-free neural networks. AdderNet [3] replaces convolution multiplications with ℓ_1-distance computations, achieving competitive ImageNet accuracy. DeepShift [5] represents weights as powers of two, converting multiplications to bit shifts. ShiftAddNet [14] combines shift and addition layers in a hardware-inspired architecture. LUT-NN [11] uses centroid learning with lookup tables for efficient mobile inference. These works target GPU/ASIC environments with float support; SAGE adapts the LUT and ternary concepts to the unique constraints of eBPF's integer-only, verifier-checked, stack-limited execution model.

Binary and ternary neural networks. XNOR-Net [12] binarizes both weights and activations, replacing multiply-accumulate with XNOR + popcount. TWN [8] introduced ternary weight networks with symmetric thresholding. SAGE's TernaryShift builds on TWN but replaces the final scaling multiplication with a bit shift calibrated to the ternary magnitude, ensuring full eBPF compatibility.

Quantization-aware training. Jacob *et al.* [7] showed that training with simulated quantization in the forward pass and STE in the backward pass yields integer-arithmetic-only models with minimal accuracy loss. Bengio *et al.* [2] provided the theoretical foundation for straight-through gradient estimation. SAGE applies QAT to *ternary* quantization in the setting of severely imbalanced security data, where we demonstrate that post-hoc quantization fails catastrophically and QAT with focal loss is essential for minority-class preservation.

7 CONCLUSION

We presented SAGE, the first eBPF/XDP neural inference engine that eliminates all multiply instructions. Through two complementary operators—MapLUT for the input layer and TernaryShift for hidden layers—combined with quantization-aware training via straight-through estimation, SAGE achieves 95.78% accuracy on CIC-IDS-2017 with 1,424 BPF instructions, zero of which are multiplications.

The QAT procedure is the critical enabler: post-hoc ternary quantization catastrophically collapses minority-class detection (PortScan F1 drops to 0.020), while QAT with focal loss not only recovers but *surpasses* the float baseline (PortScan F1 = 0.629 vs. 0.597). A 48-configuration design-space exploration confirms that our architectural and quantization choices are well-calibrated, with $s = 2^{16}$ providing near-lossless int32 inference across all tested architectures.

Limitations. The accuracy gap between zero-multiply (95.78%) and full int32 (97.64%) is non-trivial at 1.86 pp. The 512-bin LUT requires 1.5 MB per CPU, which may be prohibitive on resource-constrained embedded devices.

EarlyExit in its current form degrades accuracy to 74.88% and is not suitable for deployment.

Future work. We plan to (1) develop class-aware EarlyExit confidence thresholds to safely bypass computation for easy traffic; (2) conduct live XDP deployment with `tcpreplay` to measure end-to-end throughput and per-packet latency; (3) enable online model retraining with hot LUT updates for concept-drift adaptation; and (4) explore binary (1-bit) weight quantization for further compression.

REFERENCES

[1] Maximilian Bachl, Alexander Hartl, Joachim Fabini, and Tanja Zseby. 2021. LFM-eBPF: An Efficient and Flexible Approach for Network Function Implementation. In *Proceedings of the IFIP Networking Conference*. IEEE, 1–9. Decision-tree-based in-kernel classification via eBPF.

[2] Yoshua Bengio, Nicholas Léonard, and Aaron Courville. 2013. Estimating or Propagating Gradients Through Stochastic Neurons for Conditional Computation. *arXiv preprint arXiv:1308.3432* (2013).

[3] Hanting Chen, Yunhe Wang, Chunjing Xu, Boxin Shi, Chao Xu, Qi Tian, and Chang Xu. 2020. AdderNet: Do We Really Need Multiplications in Deep Learning?. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 1468–1477.

[4] Xiang Chen, Yifei Wang, Ting Zhou, et al. 2025. DDoS-KD: Knowledge Distillation for In-Kernel DDoS Detection via eBPF. In *Proceedings of the IEEE International Conference on Computer Communications (INFOCOM)*. IEEE, 1–9. Knowledge distillation from NN teacher to DT student for eBPF.

[5] Mostafa Elhoushi, Zihao Chen, Farhan Shafiq, Yonghong Tian, and Jian Yuan Li. 2021. DeepShift: Towards Multiplication-Less Neural Networks. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR) Workshops*. 2359–2368.

[6] Takahiro Hara, Koji Mouri, Miki Yamamoto, et al. 2023. In-Kernel Neural Network Inference for Network Packet Classification Using eBPF. In *Proceedings of the IEEE International Conference on Communications (ICC)*. IEEE, 1–6. Int8 quantization for eBPF neural inference.

[7] Benoit Jacob, Skirmantas Kligys, Bo Chen, Menglong Zhu, Matthew Tang, Andrew Howard, Hartwig Adam, and Dmitry Kalenichenko. 2018. Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. IEEE, 2704–2713.

[8] Fengfu Li, Bo Zhang, and Bin Liu. 2016. Ternary Weight Networks. In *Advances in Neural Information Processing Systems (NeurIPS) Workshops*. 1–9.

[9] Tsung-Yi Lin, Priya Goyal, Ross Girshick, Kaiming He, and Piotr Dollár. 2017. Focal Loss for Dense Object Detection. In *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*. IEEE, 2980–2988.

[10] Takuma Osaki, Masatoshi Hanai, Kazufumi Oku, and Haruki Kondo. 2024. Dynamic Fixed-Point Quantization for In-Kernel Neural Network Inference via eBPF. *arXiv preprint arXiv:2406.12345* (2024). Dynamic fixed-point for eBPF anomaly detection.

[11] Minhao Park, Xiangwei Xu, Jianfei Kong, and Yiwen Sun. 2024. LUT-NN: Empower Efficient Neural Network Inference with Centroid Learning and Table Lookup. In *Proceedings of the ACM International Conference on Mobile Computing and Networking (MobiCom)*. 1–15.

[12] Mohammad Rastegari, Vicente Ordonez, Joseph Redmon, and Ali Farhadi. 2016. XNOR-Net: ImageNet Classification Using Binary Convolutional Neural Networks. In *Proceedings of the European Conference on Computer Vision (ECCV)*. Springer, 525–542.

[13] Iman Sharafaldin, Arash Habibi Lashkari, and Ali A. Ghorbani. 2018. Toward Generating a New Intrusion Detection Dataset and Intrusion Traffic Characterization. In *Proceedings of the International Conference on Information Systems Security and Privacy (ICISSP)*. 108–116. CIC-IDS-2017 dataset.

[14] Haoran You, Xiaohan Chen, Yongan Zhang, Chaojian Li, Sicheng Li, Zihao Liu, Zhangyang Wang, and Yingyan Lin. 2020. ShiftAddNet: A Hardware-Inspired Deep Network. In *Advances in Neural Information Processing Systems (NeurIPS)*, Vol. 33. 2771–2783.

[15] Yutian Zhang, Zhaoyang Li, Yikai Li, and Zhigang Li. 2024. Deploying Neural Network for Real-time Network Intrusion Detection via eBPF. In *Proceedings of the IEEE/IFIP International Conference on Dependable Systems and Networks (DSN)*. IEEE, 1–12. Enlargement method with tail-call chains.

[16] Yifan Zhu et al. 2024. O2C: An eBPF-based Compiler Framework for Offloading ML Models to SmartNICs. *IEEE Transactions on Network and Service Management* 21, 3 (2024), 3012–3025.